Project Palantir — Master Feature Roadmap

> **Single source of truth.** This document supersedes `FEATURES-OLD.md`, `FEATURES_v2-OLD.md`, and `FEATURE-v2-ext-OLD.md`. All earlier roadmap documents are retained for historical reference only and contain known technical inaccuracies (see §8 "Standards Alignment & Corrections"). Estimates have been intentionally removed — scheduling decisions live in the sprint plan, not here.
>
> Items are strictly ordered by execution sequence. Work at the **top of this file is started first**; work at the **bottom is finished last**. Dependencies are explicit. Each item's mathematical, physical, and protocol details have been cross-checked against the authoritative specifications listed in §9.

---

0. Baseline — Already Operational

The following capabilities are in `master` today and must not be regressed. All downstream work in §1–§6 composes on top of this baseline through documented network interfaces only.

Capability	Implementation	Verification
SGP4/SDP4 orbit propagation at 1 Hz	`OrbitPropagationService` — Orekit 12.2 `TLEPropagator` under `@Scheduled(fixedRate = 1000)`; default ISS TLE loaded in `@PostConstruct`	Visible at `INFO` logs `[ISS (ZARYA)] Position — lat=… lon=… alt=… km`
Hot-swappable TLE ingestion	`TleIngestionController` → `AtomicReference<TLEPropagator>` lock-free swap	`POST /api/orbit/tle` returns `ACTIVE`; propagation continues on next tick with zero downtime
Coordinate pipeline `TEME → ITRF → Geodetic (WGS-84)`	`OneAxisEllipsoid` with `Constants.WGS84_EARTH_EQUATORIAL_RADIUS`, `WGS84_EARTH_FLATTENING`, `IERSConventions.IERS_2010`	Unit-test-validated physical bounds: lat ∈ [−90°, +90°], lon ∈ [−180°, +180°], alt > 0 km
CCSDS Space Packet encoding (18 B)	`CcsdsTelemetrySender` — 6 B Primary Header + 12 B payload (3× IEEE 754 big-endian float), 14-bit `AtomicInteger` sequence counter, `APID = 100`	Hex dump logged on every `DEBUG` TX; wire-compatible with CCSDS 133.0-B-2 Primary Header (§9)
UDP downlink to Yamcs	`DatagramSocket` → `palantir-yamcs:10000/udp` via Docker DNS	Parameters `/Palantir/Latitude`, `/Palantir/Longitude`, `/Palantir/Altitude` update live in Yamcs Web UI
UDP uplink (telecommand receive)	`UdpCommandReceiver` — Virtual Thread executor bound on `palantir.uplink.port` (default 10001); opcode dispatch for `0x01` PING, `0x02` REBOOT_OBC, `0x03` SET_TRANSMIT_POWER	Test-profile binds to `port=0` (ephemeral) to avoid CI conflicts

Yamcs instance `palantir`	`UdpTmDataLink` (:10000), `UdpTcDataLink` → `palantir-core:10001`, `GenericPacketPreprocessor` with `seqCountOffset=2` and `useLocalGenerationTime=true`, `StreamTmPacketProvider` + `StreamParameterProvider` + `StreamTcCommandReleaser` on `realtime` processor	XTCE MDB `yamcs/mdb/palantir.xml` decodes `Palantir_Nav_Packet` (APID = 100) into three float32 parameters
XTCE commands	`PING` (OpCode `0x01`), `REBOOT_OBC` (OpCode `0x02`)	Round-trip validated: Yamcs Web UI → `UdpTcDataLink` → `UdpCommandReceiver` logs `[COMMAND RECEIVED]`
Containerized stack	`docker compose up --build` — `yamcs` (healthcheck on `GET /api/`) + `palantir-core` (`depends_on: service_started`, not `service_healthy`, to avoid permanent DNS-resolution failure on `UdpTcDataLink` init)	Named volume `palantir_yamcs_data` persists the archive
Test coverage	JUnit 5 + Mockito + AssertJ — `TleIngestionControllerTest` (`@WebMvcTest`), `OrbitPropagationServiceTest` (`@SpringBootTest` + `ArgumentCaptor<Float>` physical-bounds assertions), `PalantirApplicationTests` (context load)	`mvn test` green; JaCoCo report at `target/site/jacoco/index.html`

> **Core-isolation directive.** All feature work below is developed in **independent projects** (separate Maven modules, standalone Python scripts, independent Spring Boot microservices, Yamcs plugin JARs) that interact with the baseline exclusively through UDP (`10000`, `10001`), the Yamcs REST API (`:8090/api`), the Yamcs WebSocket API (`:8090/api/websocket`), and the shared XTCE Mission Database. No pull request modifying `src/main/java/io/github/jakubt4/palantir/` is accepted for the student programme. This is the same integration model required for a commercially viable multi-tenant GSaaS offering.

---

1. Phase A — Operator HMI & Analytics Foundation *(start here)*

Why this is first. The HMI and analytics tier are the only pieces that consume the baseline telemetry pipeline without requiring any new packet definitions. They validate that the Yamcs WebSocket and Archive APIs behave as advertised, produce visible deliverables for the first sprint review, and unblock downstream integration testing.

1.1 PAL-101 — Real-time orbital ground track on CesiumJS

- **Objective.** Browser-based 3D operator display that subscribes to `/Palantir/Latitude`, `/Palantir/Longitude`, `/Palantir/Altitude` via the Yamcs WebSocket endpoint (`ws://localhost:8090/api/websocket`, processor `realtime`, instance `palantir`) and renders the spacecraft ground track on a CesiumJS globe.

- **Technical contract.**
  - Convert geodetic `(lat°, lon°, alt_km)` to ECEF using `Cesium.Cartesian3.fromDegrees(lon, lat, alt_m)` — note: **CesiumJS expects altitude in metres, Yamcs parameter is in kilometres**. Multiply by `1000.0` at the boundary; this is the single most common visualisation bug.
  - Maintain a `SampledPositionProperty` for the spacecraft entity so the globe interpolates smoothly between 1 Hz telemetry ticks.
  - Ground track polyline retains the last **one full orbital period** (≈ 5554 s for a nominal ISS orbit) of history. Exact period is derivable from the current TLE mean motion but a fixed 93-minute window is acceptable for PoC.
  - WebSocket auto-reconnect with exponential backoff on close/error; dropped connections must not leave the entity stuck on the last-known-good position.
- **Definition of done.** Spacecraft icon moves at 1 Hz, ground track polyline visibly accumulates, altitude overlay shows 2-decimal km, `npm start` or `index.html` runs standalone.
- **Dependencies.** Palantir Core baseline only.

1.2 PAL-201 — Telemetry export & trend analysis *(parallel with 1.1)*

- **Objective.** Python pipeline that extracts archived telemetry from the Yamcs Archive API via the `yamcs-client` library and produces publication-quality flight-dynamics artifacts.
- **Technical contract.**
  - `YamcsClient('localhost:8090').get_archive('palantir').list_parameter_values('/Palantir/Latitude', start=…, stop=…)` — each record carries `.generation_time` (`datetime`) and `.eng_value` (`float`).
  - Outputs: (a) `telemetry_export.csv` with columns `timestamp, latitude_deg, longitude_deg, altitude_km`; (b) `ground_track.png` using `cartopy` PlateCarree projection with coastline overlay; (c) `altitude_profile.png` — altitude vs. time; (d) summary statistics (`min`, `max`, `mean`, `std`) printed to stdout.
  - CLI contract: `--start`, `--stop` in ISO 8601; default window `now - 2h → now`.
- **Definition of done.** `python orbital_analysis.py --start 2026-04-13T00:00:00Z --stop 2026-04-13T02:00:00Z` produces all three artifacts against a live Yamcs archive containing at least 2 h of data.
- **Dependencies.** Palantir Core baseline; at least 2 h of archived telemetry.

1.3 PAL-102 — Telecommand control panel with execution feedback

- **Objective.** Web panel that issues `PING` and `REBOOT_OBC` via the Yamcs REST commanding API and displays command history.
- **Technical contract.**
  - Issue command: `POST /api/processors/palantir/realtime/commands/Palantir/{commandName}` with an empty JSON argument body (both commands have fixed `initialValue` opcodes already declared in `palantir.xml`).
  - History: `GET /api/archive/palantir/commands` returns command records with `commandId`, `generationTime`, `commandName`, `assignments`, and a `status` field that transitions `QUEUED → RELEASED → SENT` under the current (baseline) configuration — there is no closed-loop verifier yet, so terminal state is `SENT`, not `COMPLETED`.
  - CORS is already enabled in `yamcs.yaml` (`allowOrigin: "*"`) — no proxy is needed for local development.
- **Definition of done.** Panel shows two command buttons; clicks issue HTTP `POST`; a command log refreshes every 5 s with the last 20 entries; `4xx` / `5xx` errors render inline.

- **Dependencies.** Palantir Core baseline.

1.4 PAL-202 — AOS/LOS pass prediction report *(parallel with 1.3)*

- **Objective.** Post-process archived lat/lon telemetry against a ground-station visibility mask and produce a pass-prediction CSV + visibility timeline.
- **Verified geometric model (spherical Earth, fit-for-purpose for PoC).** Let the ground station have geodetic latitude `ϕ_gs`, longitude `λ_gs`, and altitude `h_gs`, and the sub-satellite point `(ϕ_ss, λ_ss)` with spacecraft altitude `h_sat` above the ellipsoid. The Earth central angle `γ` between the station and the sub-satellite point is computed via the Haversine formula:

  ```
  a = sin²((ϕ_ss − ϕ_gs) / 2) + cos(ϕ_gs) · cos(ϕ_ss) · sin²((λ_ss − λ_gs) / 2)
  γ = 2 · atan2(√a, √(1 − a))
  ```

  The elevation angle above the station's local horizon is then:

  ```
  el = atan2( cos(γ) − R_E / (R_E + h_sat),  sin(γ) )
  ```

  where `R_E ≈ 6371.0 km`. `el` crossing above `el_min` (typically 5°–10°) defines AOS; crossing back below defines LOS. This matches the standard flight-dynamics derivation and is what every operational scheduler implements before moving to a fully ellipsoidal model.
- **Default ground station. Banská Bystrica (Univerzita Mateja Bela)** — `48.7363°N, 19.1462°E, 346 m` — the project's default student recruitment source; the station is configurable via CLI for other sites.
- **Definition of done.** `pass_report.csv` with `pass_number, aos_time, los_time, max_elevation_deg, duration_seconds`; `visibility_timeline.png` showing `el(t)` with AOS/LOS markers; validated against ≥ 6 h of archive producing ≥ 2 passes for an ISS-class LEO orbit.
- **Dependencies.** Palantir Core baseline; at least 6 h of archived telemetry.

---

2. Phase B — Mission Database Expansion

Why this is next. Every subsequent payload, command, and algorithm integration inherits the XTCE extension pattern established here. Phase B is the architectural gatekeeper for Phases C–E. It must land and be validated end-to-end before the first plugin or simulator ticks a UDP packet.

2.1 PAL-301 — XTCE environmental payload definition (APID 200)

- **Objective.** Extend `yamcs/mdb/palantir.xml` with a new `Env_Payload_Packet` container restricted to `APID = 200`, inheriting from the existing abstract `CCSDS_Packet_Base`.

- **Parameter set (verified ranges).**
  - `Board_Temperature` — `float32_t`, °C. `DefaultAlarm/StaticAlarmRanges`: **watch** ≥ +60 °C, **critical** ≥ +80 °C. These thresholds bracket the typical COTS electronics ratings for LEO CubeSat EEE parts.
  - `Battery_Voltage` — `float32_t`, V. Alarms: **watch** ≤ 7.0 V, **critical** ≤ 6.0 V (standard 2S Li-ion brown-out regime).
  - `Solar_Panel_Current` — `float32_t`, A. Alarms: **watch** ≤ 0.1 A (eclipse plausible), **critical** ≤ 0.0 A (panel/string failure).
  - `Radiation_Dose` — `uint16_t` raw counts, with a `PolynomialCalibrator` mapping `dose_mrad = counts · 0.0472 + 0.5`. `<PolynomialCalibrator>` must carry both `<Term coefficient="0.5" exponent="0"/>` and `<Term coefficient="0.0472" exponent="1"/>`.
- **XTCE pattern.** Clone the existing `<SequenceContainer name="Palantir_Nav_Packet">` block, change the `RestrictionCriteria` comparison value from `100` to `200`, replace the `<EntryList>` with the four parameter references, and append one `<FloatParameterType>` per sensor carrying the `DefaultAlarm` element.
- **Definition of done.** File validates against the Yamcs XTCE loader at startup; all four parameters browsable in Yamcs Web UI; alarm ranges visible in the parameter detail view.
- **Dependencies.** Palantir Core baseline.

2.2 PAL-302 — Multi-packet stream validation

- **Objective.** Prove end-to-end data flow for APID 200 by injecting byte-level test packets and verifying ingest, decode, alarm triggering, and archive retrieval.
- **Technical contract.**
  - Python test sender `send_env_packet.py` — uses `struct.pack('>HHH' + 'fffH', packet_id, seq, length, temp, voltage, current, rad_counts)` with big-endian byte order. `packet_id` must encode `Version=000 | Type=0 | SecHdr=0 | APID=200` → `0x00C8`; `seq` uses grouping flag bits `11` and a 14-bit counter (`0xC000 | n`); `length = payload_bytes - 1`.
  - Reference the existing `CcsdsTelemetrySender.java` (read-only) for the canonical byte layout.
  - Yamcs requires no new data link: `UdpTmDataLink` on port 10000 ingests all APIDs; the `GenericPacketPreprocessor` routes to `tm_realtime`; XTCE container restriction demultiplexes by APID.
- **Definition of done.** Injecting a test packet with `Board_Temperature = 85.0` raises the **critical** alarm in Yamcs; `yamcs-client list_parameter_values('/Palantir/Board_Temperature')` returns the archived value; `validation_report.md` captures the test procedure with UI screenshots.
- **Dependencies.** §2.1 (PAL-301) merged into the MDB.

---

3. Phase C — Enterprise Java & Yamcs Extensions *(air-gapped projects)*

Why this is next. Phase C exercises the extension points a commercial operator actually leans on — server-side derived parameters, independent payload streams, and CI-grade integration testing — under the strict core-isolation constraint. All three deliverables ship as **independent Maven modules** that import nothing from `io.github.jakubt4.palantir`.

3.1 PAL-401 — Yamcs algorithm plugin (quaternion → Euler)

- **Objective.** Java plugin that consumes four raw attitude quaternion parameters `(q0, q1, q2, q3)` from an `Attitude_Packet` (APID 300) and publishes derived `Roll_deg`, `Pitch_deg`, `Yaw_deg` inside the Yamcs realtime processing pipeline.
- **Plugin model.** Implement `org.yamcs.algorithms.AbstractAlgorithmExecutor`. The plugin JAR is built in an independent Maven project (e.g. `yamcs-plugins/attitude-processor/`) with a pinned `org.yamcs:yamcs-api` dependency matching the running server (5.12.x). Mount the JAR at `/opt/yamcs/lib/ext/` via a Yamcs Dockerfile overlay and reference the algorithm from XTCE with a `<CustomAlgorithm>` entry.
- **Verified Tait-Bryan conversion (ZYX aerospace convention, `q0 = q_w`).** From a normalised unit quaternion:

  ```
  Roll  (ϕ) = atan2( 2·(q0·q1 + q2·q3),  1 − 2·(q1² + q2²) )
  Pitch (θ) = asin ( 2·(q0·q2 − q1·q3) )
  Yaw   (ψ) = atan2( 2·(q0·q3 + q1·q2),  1 − 2·(q2² + q3²) )
  ```

  These three formulas are the ones published in the Wikipedia "Conversion between quaternions and Euler angles" reference and are the form used in virtually every aerospace autopilot codebase. The `asin` form is singular at `θ = ±π/2` (gimbal lock); the unit test vectors **must** include a pitch = +90° case and verify graceful degradation (roll and yaw become coupled; the algorithm logs a `WARNING` event when `|2·(q0·q2 − q1·q3)| ≥ 1 − 10⁻⁶`).
- **Norm check.** The algorithm must log a Yamcs event at `WARNING` severity when `||q|| = √(q0² + q1² + q2² + q3²)` deviates from `1.0` by more than `1 × 10⁻⁴` — this is the defensive guard against corrupted attitude data.
- **Definition of done.** Independent Maven build; plugin loads at Yamcs startup without classloader errors; Euler outputs validated against at least five canonical reference quaternions (including one gimbal-lock case); JUnit 5 coverage > 90 % on the math class; Python test harness `send_attitude_packet.py` injects known quaternions via UDP 10000.
- **Dependencies.** Palantir Core baseline; benefits from §2.1's XTCE pattern but is not strictly blocked on it.

3.2 PAL-402 — Independent EO payload simulator (APID 400) *(parallel with 3.1)*

- **Objective.** Standalone Spring Boot 3.x microservice that simulates an Earth-observation sensor suite and streams CCSDS packets with **APID 400** to the shared Yamcs instance.
- **Packet payload (26 B total: 6 B header + 20 B payload).**
  - `Band_Red` — `float32`, reflectance ∈ [0.0, 1.0]
  - `Band_Green` — `float32`, reflectance ∈ [0.0, 1.0]
  - `Band_NIR` — `float32`, reflectance ∈ [0.0, 1.0]
  - `CCD_Temperature` — `float32`, °C ∈ [−40.0, +60.0]
  - `Shutter_State` — `uint16` ∈ {0 = CLOSED, 1 = OPEN, 2 = CALIBRATING}
  - 2 B padding (reserved)
- **Physical model.** Band intensities modulate as `max(0, sin(2π · t / T_orbit))` tied to the simulated sun illumination angle, with `T_orbit ≈ 5554 s` (ISS). CCD temperature drifts between eclipse (−20 °C) and sunlit (+45 °C) via a low-pass filter on the illumination signal. Shutter transitions to `OPEN` when illumination > 0.05, back to `CLOSED` in eclipse.

- **Deployment.** Independent Maven project `simulators/eo-payload-sim/` with its own `pom.xml`, `Dockerfile`, and `application.yaml`. Joins the compose network as `eo-payload-sim`. Byte order is **big-endian** — use `ByteBuffer.allocate(26).order(ByteOrder.BIG_ENDIAN)`; byte-order bugs are the single most common failure mode in CCSDS encoder work.
- **XTCE.** New `EO_Payload_Packet` container (APID 400) in `palantir.xml` with the five parameter references, units, and `DefaultAlarm/StaticAlarmRanges` on CCD_Temperature (**watch** ≥ +50 °C, **critical** ≥ +55 °C).
- **Definition of done.** 1 Hz packets visible in Yamcs Web UI; eclipse thermal cycle observable over one orbital period; `docker compose up eo-payload-sim` starts the service alongside the baseline stack with no port conflict.
- **Dependencies.** Palantir Core baseline; XTCE pattern from §2.1 (PAL-301) recommended.

3.3 PAL-403 — Testcontainers stress validation

- **Objective.** Automated integration suite (independent Maven module `integration-tests/`) that spins up the full Docker stack and injects 10 000 CCSDS packets to validate end-to-end integrity.
- **Technical contract.**
  - Use Testcontainers 2.x `ComposeContainer` (Compose V2). `DockerComposeContainer` is V1-only and is deprecated by Docker and marked legacy by Testcontainers.
  - Wait strategies: HTTP 200 on `GET http://yamcs:8090/api/` **plus** a secondary readiness poll on `GET /api/processors/palantir/realtime` until `"state": "RUNNING"` — Yamcs passes its basic health check **before** the realtime processor is ready to ingest, and ignoring this race is the #1 cause of flaky Testcontainers runs.
  - Inject packets via a plain `DatagramSocket` with monotonically increasing `latitude = packetIndex · 0.001f` so completeness can be validated by sampling, not by per-packet assertions.
  - Assertions: (a) ≥ 9 950 parameter values archived (< 0.5 % UDP loss is acceptable on localhost), (b) no sequence-counter gap larger than 5, (c) sampled engineering values match the injected payload within IEEE 754 float32 precision.
  - Additional cases: **malformed packet** (4-byte truncated) — Yamcs must survive and continue; **multi-APID demux** — interleaved APID 100 / APID 200 streams — each parameter set archived independently (requires PAL-301 MDB present).
- **CI contract.** Produces Surefire/Failsafe XML under `target/failsafe-reports/` (use `maven-failsafe-plugin`, not surefire, for integration tests) and a `stress_test_report.json` capturing `total_packets_sent`, `total_packets_archived`, `loss_percentage`, `injection_duration_ms`, `packets_per_second`, `archive_query_latency_ms`.
- **Definition of done.** `mvn verify` green in under 5 minutes (excluding image pull); reports parseable by Jenkins/GitHub Actions. This is the CI quality gate for ECSS-Q-ST-80C Rev.2 §6.2.4 software verification evidence.
- **Dependencies.** Palantir Core baseline; multi-APID case depends on §2.1 (PAL-301); otherwise independent.

---

4. Phase D — Space Situational Awareness & Collision Avoidance *(Demo Day 2026-09-29 bundle)*

Why this is next. Phases A–C have hardened the ingestion chain; Phase D introduces active safety-critical automation. PAL-501 + PAL-502 are the two deliverables bundled for Demo Day 2026-09-29 together with the THRUST_MANEUVER wiring in §4.3. Every item in this phase is safety-adjacent; none of it should be attempted

before the Phase C integration tests are green.

4.1 PAL-501 — Conjunction assessment with Monte Carlo Pc

- **Objective.** Standalone Orekit 12.2 batch application (`hpc/conjunction-assessment/`) that screens the Palantir spacecraft against the CelesTrak GP catalog and computes Time of Closest Approach (TCA) and Probability of Collision (Pc) for each close approach.
- **Catalog ingestion.** CelesTrak GP API endpoint: `https://celestrak.org/NORAD/elements/gp.php?GROUP=active&FORMAT=…`. **Prefer OMM (XML)** over TLE — CelesTrak has announced that NORAD 5-digit catalog numbers will exhaust around **2026-07-20**, after which new objects receive 6-digit catalog numbers that cannot be represented in the legacy TLE format. Orekit 12.2 ingests OMM via `OmmParser`; for 5-digit-only legacy data, `TLEParser` remains usable. The CLI must accept `--catalog-format={omm,tle}` with OMM as the default.
- **Screening stage.** For each debris object, propagate both spacecraft and debris with `TLEPropagator.selectExtrapolator(tle)` or an SGP4-based `SgpPropagator` for OMM, over a 7-day prediction window at 60-second steps. Flag any pair where min Cartesian distance (in the TEME frame, converted consistently) falls below 10 km. 60-second sampling will miss encounters faster than ≈ 150 m/s closure × 60 s = 9 km in half a step — that is exactly why the flagged pairs are refined with Monte Carlo in the next stage; the screening cadence is deliberately coarse.
- **Monte Carlo Pc estimation.** For each flagged pair, draw `N` (default 10 000) sample state vectors from a diagonal covariance (100 m 1σ position, 0.01 m/s 1σ velocity). Propagate each sample pair to its TCA. `Pc = (# samples with miss distance < R_HBR) / N` where `R_HBR = 10 m` is the combined hard-body radius (operator-configurable). Seed the RNG for reproducibility.
- **Parallelism.** Use Java 21 Virtual Threads (`Executors.newVirtualThreadPerTaskExecutor()`) across pairs — propagation is CPU-bound, not I/O-bound, but the virtual-thread model still simplifies the fan-out without measurable penalty for this scale.
- **Local validation profile (mandatory before HPC).** `--catalog-limit=500 --prediction-hours=24 --mc-samples=1000` must complete in < 10 minutes on a developer workstation (4 cores, 16 GB RAM) and produce a reproducible `conjunction_report.json`. **No Slurm access is granted until the local profile is green.**
- **Yamcs integration.** Top-10 conjunctions (by `Pc`) posted as events via `POST /api/archive/palantir/events` with severity mapped from `Pc`: `WARNING` for `1e-6 ≤ Pc < 1e-4`, `CRITICAL` for `Pc ≥ 1e-4`. The 1e-4 threshold is the industry-standard COLA "red" trigger used by NASA CARA and the ESA Space Debris Office.
- **Definition of done.** Independent Maven module; `mvn package` produces an executable JAR; `conjunction_report.json` lists each flagged conjunction with `{debris_norad_id, debris_name, tca_utc, miss_distance_m, probability_of_collision, relative_velocity_m_s}`; unit tests cover TLE/OMM parsing, pairwise distance, and Monte Carlo sampling with > 80 % line coverage on the algorithm classes.
- **Dependencies.** Palantir Core baseline.

4.2 PAL-502 — Automated collision-avoidance loop with manual operator gate

- **Objective.** Closed-loop collision-avoidance workflow that monitors conjunction events from §4.1, computes an avoidance manoeuvre when `Pc > 1e-4`, queues a `FIRE_THRUSTER` command, and **requires explicit operator approval** before release — no autonomous manoeuvre ever flies.

- **XTCE command definition.** Add `FIRE_THRUSTER` `MetaCommand` to `palantir.xml` with:
  - `opcode` (uint8, fixed = `0x04`)
  - `delta_v_x`, `delta_v_y`, `delta_v_z` — `float32` m/s in the radial-tangential-normal (RTN) frame
  - `burn_duration_s` — `uint16` seconds
  - A `DefaultSignificance` element set to `consequence="critical"`.
- **Manual-approval mechanism — factual correction vs. earlier docs.** Yamcs does **not** provide an XTCE `ManualVerifier` element; the earlier roadmap's reference to one is incorrect (see §8 "Standards Alignment & Corrections"). The operational pattern the Yamcs command-processing model actually supports for manual release is:
  1. **Command significance** — declare `<DefaultSignificance consequenceLevel="critical"/>` on the `MetaCommand`. This raises the significance level carried on the command object.
  2. **Command queue with `minLevel: critical`** — configure a dedicated queue in `yamcs.palantir.yaml` `commandQueues:` with `minLevel: critical` and `state: BLOCKED` (or an equivalent `manual`-release mode depending on Yamcs minor version). Commands of significance ≥ critical enter the queue and remain there until an operator performs an explicit "release" action through the Yamcs Web UI / REST API (`POST /api/queue/…/release-command`).
  3. Optional **TransmissionConstraint** against a `FIRE_THRUSTER_ARMED` ground-parameter to belt-and-brace the gate — this parameter is flipped by the operator pressing a dedicated "ARM" toggle and auto-resets after 60 s.

  The script **must never** attempt to bypass the queue. If the queue's minLevel is lowered or `state` flipped to `ENABLED` below `critical`, the Python script aborts with a CRITICAL log message.
- **Delta-v computation (simplified PoC model).** `delta_v_tangential = 0.01 m/s · clamp(Pc / 1e-4, 1, 10)`; radial and normal components zero; `burn_duration_s = 10`. This is intentionally scalar — the goal is to exercise the closed-loop workflow, not to produce a flight-quality manoeuvre; production systems use the full Lambert/CW differential corrector, which is explicitly out of scope.
- **Script behaviour.** `collision_avoidance.py` polls `GET /api/archive/palantir/events` on a 30 s cadence, filters `severity=CRITICAL AND source='conjunction_assessment'`, issues `POST /api/processors/palantir/realtime/commands/Palantir/FIRE_THRUSTER` with the computed delta-v, and then polls `GET /api/processors/palantir/realtime/commands/{commandId}` for state transitions. Terminal states it must handle: `QUEUED → ACCEPTED → SENT` (approved path), or `REJECTED` (operator denial).
- **Definition of done.** End-to-end test: inject a synthetic conjunction event with `Pc = 5e-4`; observe the script enqueue the command; confirm the command remains blocked until an operator clicks "release" in the Yamcs Web UI; verify `SENT` reaches the `UdpTcDataLink`; `COLA_PROCEDURE.md` documents the full workflow including the abort paths.
- **Dependencies.** §4.1 (PAL-501) conjunction events; XTCE schema familiarity from §2.
- **Also required for Demo Day bundle.** §4.3 wiring.

4.3 Phase 3b — THRUST_MANEUVER physics reaction wiring

- **Objective.** Close the propagation loop: when a `FIRE_THRUSTER` command is released and received by `UdpCommandReceiver`, the `OrbitPropagationService` must apply a matching impulsive delta-v to the active spacecraft state and continue propagation from the perturbed orbit.
- **Strictly core-team work.** This is the one and only change to `src/main/java/io/github/jakubt4/palantir/` in the entire Demo Day bundle. Because the core is off-limits to students, §4.3 is owned by the lead engineer and is the integration piece that makes PAL-502 visible on the CesiumJS HMI (§1.1) — once the delta-v is applied, the ground track visibly shifts.

- **Technical sketch.**
  - Extend the `UdpCommandReceiver` opcode switch with `0x04 → THRUST_MANEUVER`, decoding the 14-byte payload `(Δv_x, Δv_y, Δv_z, burn_s)` in big-endian order.
  - Publish a new internal event `ThrustManeuverCommand` via a Spring `ApplicationEventPublisher`. `OrbitPropagationService` listens, converts the `AtomicReference<TLEPropagator>` to a current `SpacecraftState`, applies the impulsive delta-v in the RTN frame using Orekit's `LOFType.QSW` or equivalent local orbital frame, then builds a new `NumericalPropagator` initialised from that state (TLE is not re-used after the manoeuvre — mean elements would drift from the post-manoeuvre osculating state).
  - After the burn the propagator switches from `TLEPropagator` (SGP4) to a numerical propagator with a simple point-mass attraction + J_2 zonal term, which is the correct physical model once a TLE no longer represents the orbit.
- **Definition of done.** Operator issues `FIRE_THRUSTER` with `Δv_tangential = 5 m/s`; HMI shows ground track shift of the expected magnitude within the next orbit; a regression test asserts that post-manoeuvre altitude delta matches the analytic `2·Δv · √(a/μ)` prediction within 2 %.
- **Dependencies.** §4.2 (PAL-502). This is the final Demo Day integration piece.

---

5. Phase E — Synthetic Telemetry & ML Anomaly Detection

Why this is next. Phase E produces the training data the downstream Telemetry Intelligence product tier needs. It is deferred until after Phase D because (a) it does not unblock Demo Day, and (b) it is stylistically the cleanest standalone track — the ML pipeline never touches the flight core or the SSA loop.

5.1 PAL-503 — Synthetic telemetry generator with anomaly injection

- **Objective.** Standalone Java application (`simulators/synth-telemetry-gen/`) producing deterministic 1 Hz CCSDS telemetry on **APID 500** with runtime-configurable anomaly injection.
- **Nominal signal model (payload: 24 B).**
  - `Latitude = A · sin(2π · t / T_orbit)`, `A = 51.6°` (ISS inclination), `T_orbit = 5554 s`
  - `Longitude` — sawtooth `-180° + (360° · (t mod T_orbit) / T_orbit)` with a nodal-regression drift of `-5.0° / 5554 s`
  - `Altitude = 408.0 + 2.0 · sin(2π · t / T_orbit + π/4)` km
  - `Board_Temp` — sinusoidal between +20 °C (sunlit) and −5 °C (eclipse)
  - `Bus_Voltage = 28.0 V ± 0.3 V` white-noise ripple
  - `Gyro_Rate_Z = 0 °/s ± 0.01 °/s` noise
- **Anomaly state machine (stdin-driven).**
  - `THERMAL_RUNAWAY` — `Board_Temp` linear ramp to +95 °C over 120 s, then hold
  - `VOLTAGE_DROP` — `Bus_Voltage` exponential decay `28 → 18 V` with `τ = 60 s`
  - `TUMBLE` — `Gyro_Rate_Z` linear ramp `0 → 15 °/s` over 30 s
  - `NOMINAL` — reset to the default model
  - Single-active-anomaly model: `NOMINAL` is implicit between any two injections. Simultaneous anomalies are out of scope for the first pass.

- **Dataset recording.** `--record synth_telemetry.csv` appends rows `timestamp_ms, latitude, longitude, altitude, board_temp, bus_voltage, gyro_rate_z, anomaly_label`. A 1-hour recording with two injected anomalies must produce ≥ 3 600 labelled rows.
- **XTCE.** `Synth_Telemetry_Packet` container (APID 500) with all six parameters and alarm ranges matching the injection thresholds.
- **Dependencies.** Palantir Core baseline; XTCE pattern from §2.1.

5.2 PAL-504 — LSTM autoencoder training & ONNX export

- **Objective.** PyTorch unsupervised anomaly detector trained on §5.1's nominal data, exported to ONNX for portable inference.
- **Architecture.**
  - Input: sliding window of 60 samples × 6 features (60 s at 1 Hz)
  - Encoder: `LSTM(input_size=6, hidden_size=32, num_layers=2, batch_first=True)` → final hidden state
  - Bottleneck: `Linear(32 → 8) → ReLU → Linear(8 → 32)`
  - Decoder: `LSTM(32 → 32, 2 layers) → Linear(32 → 6)`
  - Loss: MSE between input window and reconstruction
- **Training & calibration.** Adam, `lr = 1e-3`, `batch_size = 64`, ≤ 100 epochs with early stopping (patience = 10) on NOMINAL-only validation loss. Threshold `T` = 99th percentile of reconstruction error on NOMINAL validation; anomaly flag when error > `T`.
- **ONNX export.** `torch.onnx.export(..., dynamic_axes={'input': {0: 'batch'}, 'output': {0: 'batch'}})`. Validate the exported model under `onnxruntime` on 10 test windows (5 nominal, 5 anomalous) and assert outputs match PyTorch within `1e-5` absolute tolerance. Silent numerical drift between PyTorch and `onnxruntime` is a well-known edge case for LSTMs — the validation step is mandatory, not optional.
- **Evaluation targets.** Per-anomaly-type detection rate > 95 %; false positive rate < 2 % on the NOMINAL validation subset.
- **Deliverables.** `anomaly_detector.onnx`, `scaler_params.json`, `training_report.json`, `reconstruction_error.png`.
- **Dependencies.** §5.1 (PAL-503) dataset.

---

6. Phase F — Future Concepts *(finish last — intentionally deferred)*

Items below are on the long-term backlog and are intentionally unsequenced. They are listed in rough order of expected value but are not scheduled against any milestone.

1. **Ground Station Network Service.** Parameterised AOS/LOS scheduler fed by a ground-station inventory (Banská Bystrica UMB, ESA ESTRACK Kiruna/Redu/New Norcia reference data for comparison). Builds on §1.4 PAL-202 geometry with an ellipsoidal Earth model and refraction correction.
2. **CCSDS SDLS (Space Data Link Security).** HMAC authentication on telecommand packets per CCSDS 355.0-B-2. Threat model to be documented before any implementation; SDLS is non-trivial and should not be built without a real adversary model.
3. **Multi-satellite constellation support.** Replace the single `AtomicReference<TLEPropagator>` with a keyed map and per-spacecraft APIDs. Virtual Thread per spacecraft already maps cleanly onto this.
4. **Kubernetes / Helm chart deployment.** Replace `docker compose` with a Helm chart for multi-tenant GSaaS operation. Horizontal scaling applies only to the propagation tier; Yamcs is a singleton per instance.

5. **Orekit 13.x upgrade.** Current code uses Orekit 12.2; Orekit 13.1.4 (released 2026-02-08) is the latest. Upgrade is low risk — API surface used by the project (`TLE`, `TLEPropagator`, `OneAxisEllipsoid`, `IERSConventions`) is stable — but touches the flight-critical core and so is deferred to a quiet sprint with dedicated regression testing.
6. **Yamcs 5.12.5 / 5.13 upgrade.** Currently pinned at 5.12.2. The version drift is minor; defer until the Yamcs 5.13 line stabilises and the plugin API (§3.1) is known to remain source-compatible.
7. **CCSDS 133.0-B-2 clarifications adoption.** Wire format is already compatible (Primary Header unchanged between B-1 and B-2); adopt the B-2 terminology and idle-packet handling in the encoder after 5.12 upgrade.
8. **ESA BIC technical readiness package.** Predictive Orbital Shadowing (cross-check Orekit ideal state vs. ingested parameters, fire `ParameterAlarm` on divergence) and closed-loop command verification (bidirectional ACK) as presented in the earlier `FEATURES_v2-OLD.md`. These are valuable marketing stories but depend on Phases C + D being stable in production, which is why they sit at the bottom.

---

7. Dependency Graph (topological, top-to-bottom)

```
Baseline (§0)
 │
 ├── PAL-101  (§1.1)  HMI ground track
 ├── PAL-201  (§1.2)  analytics pipeline     ‖ parallel
 ├── PAL-102  (§1.3)  command panel          ‖ parallel
 └── PAL-202  (§1.4)  AOS/LOS report         ‖ parallel
       │
       ▼
 ├── PAL-301  (§2.1)  XTCE env payload  ◄── architectural gatekeeper
 └── PAL-302  (§2.2)  stream validation ◄── hard-blocked on PAL-301
       │
       ▼
 ├── PAL-401  (§3.1)  attitude plugin
 ├── PAL-402  (§3.2)  EO simulator       ‖ parallel with PAL-401
 └── PAL-403  (§3.3)  Testcontainers QA  ‖ multi-APID case blocked on PAL-301
       │
       ▼
 ├── PAL-501  (§4.1)  conjunction assessment
 ├── PAL-502  (§4.2)  COLA loop           ◄── hard-blocked on PAL-501
 └── Phase 3b (§4.3)  THRUST_MANEUVER     ◄── hard-blocked on PAL-502 + core-team owned
       │                                      ⇒ Demo Day 2026-09-29 bundle complete
       ▼
 ├── PAL-503  (§5.1)  synthetic generator
 └── PAL-504  (§5.2)  autoencoder + ONNX  ◄── hard-blocked on PAL-503
       │
       ▼
 └── Phase F  (§6)    deferred future concepts
```

Bars (`‖`) mark siblings that can run in parallel; arrows (`◄──`) mark hard blockers.

8. Standards Alignment & Corrections *(delta vs. earlier docs)*

The following technical items have been **corrected** in this revision relative to the retired `FEATURES-OLD.md`, `FEATURES_v2-OLD.md`, and `FEATURE-v2-ext-OLD.md`:

#	Old text	Issue	Corrected position
1	"Yamcs `ManualVerifier` stage" / "XTCE `ManualVerifier`" (old PAL-502)	Yamcs / XTCE 1.2 do not define a `ManualVerifier` element. Manual release is implemented via **command significance + command-queue minLevel with manual release** (optionally belt-and-braced by a `TransmissionConstraint` against a ground parameter).	§4.2 PAL-502 rewritten to use the correct Yamcs command-processing model.
2	"CCSDS 133.0-B-1" (cited uniformly)	133.0-B-2 (June 2020) is the current Blue Book. Primary Header wire format is unchanged, so existing encoders remain compliant.	§0, §9 reference 133.0-B-2 explicitly, noting wire compatibility.
3	"ECSS-Q-ST-80C Rev.1" (old PAL-403)	ECSS-Q-ST-80C Rev.2 (30 April 2025) cancels and replaces Rev.1.	§3.3 PAL-403 cites Rev.2.
4	"CelesTrak GP catalog in CSV / TLE format only" (old PAL-501)	CelesTrak 5-digit NORAD numbers exhaust ≈ 2026-07-20; new objects use 6-digit numbers that cannot be encoded as TLE. OMM (XML) is the forward-compatible format.	§4.1 PAL-501 defaults to OMM with `--catalog-format` override.
5	"Testcontainers `DockerComposeContainer`" (old PAL-403)	V1-only and deprecated; Testcontainers 2.x uses `ComposeContainer` against Compose V2.	§3.3 PAL-403 specifies `ComposeContainer`.
6	"Pitch = `asin(2·(q0·q2 − q3·q1))`" (old PAL-401)	Mathematically equivalent to `asin(2·(q0·q2 − q1·q3))` — the form used here — but I standardise on the Wikipedia-canonical ordering to make the gimbal-lock guard condition cleaner to read. Verbatim checked against Wikipedia "Conversion between quaternions and Euler angles".	§3.1 PAL-401 uses the canonical form and adds the `
7	Default ground station "Košice, 48.7164°N, 21.2611°E, 206 m" (old PAL-202)	Arbitrary choice not aligned with the project's UMB / Banská Bystrica student recruitment context.	§1.4 PAL-202 defaults to Banská Bystrica (UMB) `48.7363°N, 19.1462°E, 346 m`; station remains configurable.
8	"Yamcs health check implies processor ready" (old PAL-403)	`GET /api/` returns 200 before the realtime processor has subscribed to `tm_realtime`.	§3.3 PAL-403 adds the secondary `"state": "RUNNING"` readiness poll.

9	"AOS/LOS uses elevation = `arctan((cos γ − R/(R+h)) / sin γ)` " (old PAL-202)	Correct, but `arctan` loses sign information; swapped to `atan2` form to handle the full range cleanly.	§1.4 PAL-202 uses `atan2( cos γ − R/(R+h), sin γ )`.
10	"Post-FIRE_THRUSTER propagation remains on SGP4 TLE" (implicit in old docs)	TLE mean elements no longer represent the orbit after an impulsive manoeuvre; the propagator must switch to a numerical model.	§4.3 Phase 3b prescribes the post-manoeuvre handoff to a `NumericalPropagator` with point-mass + J_2.

Retired documents are preserved as `*-OLD.md` beside this file and should be treated as historical context only. Do not cite them in new work.

---

9. Verified Specifications & References

All technical claims in §0–§6 have been cross-checked against the authoritative primary sources below. Citations are included here so future reviewers can verify the document mathematically, physically, and protocol-wise without re-discovering the references.

CCSDS standards

- CCSDS 133.0-B-2, *Space Packet Protocol*, Issue 2, Blue Book, June 2020 — Primary Header wire format, APID field, sequence count semantics.
- CCSDS 355.0-B-2, *Space Data Link Security Protocol* — referenced only as a future-work pointer in §6.

Astrodynamics & frames

- Orekit 12.2 API (`TLE`, `TLEPropagator`, `OneAxisEllipsoid`, `IERSConventions.IERS_2010`, `FramesFactory.getITRF`) — matches the baseline code in `OrbitPropagationService`.
- Orekit 13.1.4 release notes (2026-02-08) — future-upgrade target listed in §6.
- Wikipedia, *Conversion between quaternions and Euler angles* — Tait-Bryan ZYX formulas in §3.1 verified verbatim.
- WGS-84 constants: `a = 6378137.0 m`, `1/f = 298.257223563` — delivered by `Constants.WGS84_EARTH_EQUATORIAL_RADIUS` and `Constants.WGS84_EARTH_FLATTENING`.

Mission control & telemetry

- Yamcs 5.12 Server Manual — `UdpTmDataLink`, `UdpTcDataLink`, `GenericPacketPreprocessor`, `StreamTmPacketProvider`, `StreamTcCommandReleaser`, command queues, significance levels, algorithm plugin API.
- `org.yamcs.algorithms.AbstractAlgorithmExecutor` Javadoc — §3.1 plugin model.
- XTCE 1.2 specification (OMG SpaceSystemV1.2.xsd) — container inheritance, `RestrictionCriteria`, `DefaultAlarm / StaticAlarmRanges`, `PolynomialCalibrator`, `MetaCommand / DefaultSignificance`, `TransmissionConstraint`.

Space Situational Awareness

- CelesTrak GP API (`https://celestrak.org/NORAD/elements/gp.php`) and the GP data format notice announcing 5-digit catalog exhaustion around 2026-07-20.
- ESA Space Debris Office COLA operational practice and the `Pc > 1e-4` "red" threshold referenced in ESA Space Safety Programme briefings and NASA CARA guidance.

- Probability of collision — Monte Carlo sampling method as used by NASA/ESA conjunction assessment teams (diagonal covariance in position-velocity, hard-body-radius intersection test).

Software product assurance

- ECSS-Q-ST-80C Rev.2, *Space Product Assurance — Software Product Assurance*, 30 April 2025 — supersedes Rev.1 (15 February 2017).

Testing infrastructure

- Testcontainers for Java, *Docker Compose Module* — `ComposeContainer` (V2) vs. the deprecated `DockerComposeContainer` (V1).
- `maven-failsafe-plugin` — Maven convention for integration tests.

Ground-station geodesy

- Banská Bystrica coordinates `48.7363°N, 19.1462°E, 346 m` — used as the default ground station in §1.4 to reflect the project's UMB recruitment context.

---

10. How to use this document

- **Read it top-to-bottom.** The order is the execution order. There are no hidden dependencies between sections.
- **Cite it in pull requests and sprint reviews.** If a PR touches any of §1–§5, reference the section number in the commit message and the PR description so the roadmap trace is automatic.
- **Do not edit §0 without a flight-test.** §0 is the baseline contract; any regression must be caught by the integration suite in §3.3 before the baseline line items are modified.
- **File technical corrections in §8.** If a downstream reviewer finds a mathematical or protocol issue, it belongs in §8 with the old/new pair so the audit trail is visible.
- **When in doubt, measure twice, cut once.** This is a digital twin of a real spacecraft. Every numerical error compounds.